

Smart Ground-Truthing and Digital Biodiversity System

System Design Proposal

Unit Code: COS30049

Unit Name: COMPUTING TECHNOLOGY INNOVATION PROJECT

Tutor: Lee Sue Han, Kelvin Yong, Mark Tee, Fu Swee Tee

Group Name: Plantiful (Group 7)

#	Name	Student ID
1	Muhammad Maqeel bin Muhammad Kahfi	102782384
2	Nathan Sebastian Learmonth	102782258
3	Badrul Aliff Aiman bin Badrulmunirzaki	102778273
4	Gae Jayden MWINE	104393610
5	Ashley Wallen Anak Winston	105806559
6	Basill Agas Anak Heatley Rogers	102778888

Executive Summary

Niah National Park, which is overseen by the Sarawak Forestry Corporation (SFC), contains a wide variety of plant species. These plants should be recorded and tracked for the purposes of conservation studies and ecotourism projects. Much of the work is currently carried out by man. For example, botanists take notes in the field and make recordings on paper and simple digital devices, then enter everything into the database once they return from their expedition. Conservation officers hold biodiversity knowledge across a number of disconnected sources. The result is delayed information, duplicated effort, and the risk of losing data. Rare and endangered species are also left with no automated protection against threats such as poaching and habitat disturbance.

This proposal presents a Smart Ground-Truthing and Digital Biodiversity System for NeuonAI and SFC. The system has three parts. A mobile application with QR tags allows botanists to collect data about plants in the wilderness without relying on the internet. The features offered by the app include identification of plants, gathering of data on different plant structures and capturing pictures along with GPS position using the device. When the device connects to the internet, all the records are uploaded to the data repository based on PostgreSQL known as Supabase. A web-based Digital Plant Knowledge System then lets conservation officers review and approve records, manage them, search and report, and publish information for researchers and the public. An IoT monitoring layer uses MQTT sensors and a dashboard to alert staff automatically when unusual activity is detected near vulnerable plant species. Security is designed in from the start through role-based access control, encrypted sensitive data, and an SSDLC-aligned vulnerability assessment using OWASP ZAP.

This document provides the foundation for the problem in Section 1.0. Section 2.0 explores project scope and requirements. Section 3.0 is about stakeholders. In Section 4.0, the three solutions are evaluated including their SWOT analysis and Project Forces. The recommended approach is Option C, a purpose-built hybrid system, and the reasons for that choice are presented in Section 4.0. Section 5.0 sets out the deliverables and a 13-week schedule aligned to the SSDLC. Sections 6.0 to 8.0 then describe the solution direction and architecture, key design decisions and development standards, quality attributes, and resources. Section 9.0 closes with the approval signatures.

Table of Contents

Executive Summary 2
1 Introduction 5
1.1 Background / Problem Description 5
1.2 Problem Statement 5
1.3 Existing Solutions 5
1.4 Motivation 5
2 Scope 6
2.1 Goals / Aims 6
2.2 Objectives 6
2.3 Constraints and Out-of-Scope Limitations 7
2.3.1 Constraints 7
2.3.2 Out-of-Scope Features 8
3 Stakeholders 9
4 Possible Solution Analysis 10
4.1 Justification for Selecting Option C 12
4.2 SWOT Analysis (Option A) 12
4.2.1 Strengths 12
4.2.2 Weaknesses 12
4.2.3 Opportunities 12
4.2.4 Threats 12
4.3 SWOT Analysis (Option B) 13
4.3.1 Strengths 13
4.3.2 Weaknesses 13
4.3.3 Opportunities 13
4.3.4 Threats 13
4.4 SWOT Analysis (Option C) 14
4.4.1 Strengths 14
4.4.2 Weaknesses 14
4.4.3 Opportunities 14
4.4.4 Threats 14
4.5 Project Forces Analysis (Force Field) 15
4.5.1 Driving forces (for building Option C) 15
4.5.2 Restraining forces (against / risks) 15
4.6 KoST Analysis (Knowledge, Skills, Technology) 15
4.6.1 Knowledge 15
4.6.2 Skills 15
4.6.3 Technology 16
5 Deliverables and Schedule 17
5.1 Deliverables 17

5.2 Schedule 17
5.3 Initial Release Schedule 18
5.3.1 Product Backlog 18
5.3.2 Project Timeline 19
6 Solution Direction 20
6.1 Overview 20
6.2 Client-Application-Data Architecture 21
6.3 Tier Summary 23
6.4 Sub-Component Technology Analysis 23
6.4.1 Mobile Framework 23
6.4.2 Web Framework 24
6.4.3 Backend and Database 24
6.4.5 Public Visitor Access 27
6.5 Key Designs 28
6.6 Development Process, Standards and Conventions 29
6.7 The Four Core Workflows 29
7 Quality Management 31
7.1 Risk Register 33
7.2 Acceptance Criteria and Test Strategy 33
8 Resources 35
8.1 Software / Tools 35
8.2 Hardware 35
8.3 Plant Data Sources 36
9 Approval Signatures 36
9.1 Project Team 36
9.2 Project Sponsor 37
10 References 37
11 Appendix A: Glossary of Terms 38
12 Appendix B: Offline-First Sync and Conflict Resolution Rules 38
13 Appendix C: Security Testing Evidence (SSDLC) 38

1 INTRODUCTION

1.1 Background / Problem Description

Located in the Niah National Park, Sarawak Forestry Corporation (SFC) is responsible for maintaining a variety of plant species that need to be documented and monitored over time for ecological purposes. At present, botanists and conservation officers are conducting this documentation using traditional methods like tagging plants in person, taking notes on paper or simple forms, and later inputting this information in computer databases in an office. This method causes delays and redundancy as well as increased chances of data losses since Niah is a complicated forest that needs to be accessed.

Apart from the field documentation process, the access to consolidated biodiversity data remains limited to the researchers, conservation officers and members of local communities and those who visit the park. As a result, plant species data, conservation statuses, and geographical information remain scattered instead of being in one place, which is detrimental to scientific research and public knowledge about conservation. At the same time, rare and endangered tree species located in the park are still endangered because there are no automated systems to monitor and warn respective authorities regarding any unusual activities in the area.

In this project, Group 7 (Plantiful) will design a system for smart ground-truthing and digital biodiversity technologies that will facilitate field data collection, store and manage information about biodiversity, and apply IoT monitoring to vulnerable plants in the park.

1.2 Problem Statement

Now, botanists do not have an integrated solution for scanning, recording, and geotagging plants while working offline in the field.Conservation officers also do not have a unified system for managing and publishing this information for researchers and the public. Moreover, there are no automated solutions that can safeguard rare or endangered species from threats like poaching or habitat disturbance in the absence of personnel.

Taken together, these issues point to a clear gap. The team proposes an integrated system that lets botanists scan QR-tagged plants and record species data offline in the field, lets conservation officers manage and publish that data through a centralised digital knowledge system, and adds IoT-based sensors that monitor the area near vulnerable plant species and alert staff to threats. This solves both the field-documentation problem and the biodiversity-protection problem described above.

1.3 Existing Solutions

Two categories of alternative approach already exist. The first is continuing with the paper-based and ad hoc digital methods SFC uses today such as recording sightings on paper or in basic spreadsheets and manually entering the data later, which is slow, duplicative, and error-prone. The second is adopting a generic, off-the-shelf field data collection tool such as Esri's ArcGIS Field Maps, Survey123, or the citizen-science app iNaturalist. These tools are mature and well supported for general field survey work, but none of them are built for QR-based specimen tagging, an offline-first sync pipeline feeding a bespoke conservation-officer approval workflow, or IoT-based threat monitoring, and none would give SFC or NeuonAI ownership of the resulting system. A full side-by-side comparison of these alternatives against a purpose-built system is presented in Section 4.0.

1.4 Motivation

This project matters now for three reasons. First, Niah National Park's rare and endangered plant species face time-sensitive threats such as poaching and habitat disturbance that today's manual, disconnected record-keeping cannot help SFC respond to quickly. Second, SFC and NeuonAI have a live opportunity to end up with a validated, ownable product rather than an ongoing dependency on a third-party tool or subscription, like NeuonAI's existing RoadPlus system. Third, delivering this

system gives Team Group 7 a concrete, full-stack, security-conscious engineering project spanning mobile, web, IoT, and SSDLC-aligned testing, and it directly satisfies COS30049's assessment requirements.

2 SCOPE

This project will provide Niah National Park with an integrated web-based and mobile biodiversity documentation infrastructure. Conservation officials will use a web-based knowledge system to manage, assess, and disseminate the species data that botanists scan from QR-tagged plants, record offline in the field, and sync to a central database. To aid in the protection of rare and endangered plant species, the platform also has an IoT-based monitoring layer and role-based security controls.

2.1 Goals / Aims

With the implementation of this system, Sarawak Forestry Corporation will be able to modernise the documentation process of plants in Niah National Park by replacing the traditional manual process with a cutting-edge digital system.

This system will allow botanists to easily scan and upload data about different plant species on mobile devices, including species identification, images, and location tracking. The system does not need internet access to function, but it will sync all the data with the main database once the internet is available. The gathered data can be also reviewed and published by conservationists using an online monitoring system. The system also uses IoT sensors to monitor environmental changes and detect unplanned activities around the tagged locations, including threats to plant species. This way, the new system will eliminate the unproductive use of time and resources involved in the current manual process and will enhance the protection of the most endangered species.

2.2 Objectives

It is not sufficient to have a vague idea of what to construct. The project must entail objectives that are not only specific, but also measurable, so you can know when each function is finished.

- A botanist should be able to scan a Q-code of a plant so that its record would show on the screen of a smart-device in less than five seconds, even when there is no Internet access.
- The app should allow a botanist to register a plant record in full (taxonomy, morphology, height, photos, GPS coordinates) without any Internet connection, storing the record until the connection is established.
- After the connection is restored, the app should immediately synchronize all records stored offline at the central database with no loss, avoiding conflicts where possible.
- The system shall automatically generate a unique QR code for every new plant record, linking directly to that record's stored details.
- Conservation officers shall be able to add, edit, delete, review, and approve plant species records through the web-based Digital Plant Knowledge System.
- The web system shall support searching and filtering species records by scientific name, common name, or conservation status.
- Conservation officers shall be able to generate and export biodiversity reports from the system, covering species records and field survey activity.
- The system shall enforce role-based access control so botanists, conservation officers, and admins can only access the functions relevant to their role.
- All personal records related to the individual along with individual review shall be secured through encryption. Test results shall pass the evaluation for vulnerability prior to submission and each incident occurring during testing shall be verified and corrected.

- IoT sensors shall gather up-to-the-minute environmental data (temperature, humidity, and motion) around tagged rare plant species, and send alerts to the administrator dashboard when an abnormal behavior occurs.
- Administrators shall be able to monitor sensor data and alerts through a centralised IoT dashboard, with historical data stored for long-term habitat condition analysis.
- Visitors shall be able to view published species records through a public, read-only web portal without needing an account, per the Public Visitor Access sub-component described in Section 6.3.5.

Functional area	Key requirement
Mobile field app	Offline-first plant record capture (QR, GPS, camera). Tagged plants are identified by scanning a stable QR record ID.
Web knowledge system	Species records management, review/approval, search and reporting for conservation officers.
Offline-to-cloud sync	Local SQLite store on the device. Stable record ID resolves against the central database once connectivity returns. Conflicts resolved to a defined strategy.
IoT monitoring	Sensors gather information about temperature, humidity, and movement around tagged rare/endangered species. Automated alerts created for exceptional activities.
Security & privacy	Access management based on roles where personal, plant, and assessment data are secured with encryption. SSDLC mapping of vulnerability assessment and remediation processes.
Public visitor access	Read-only web portal listing published species records only; no account required, per Section 6.3.5.

Table 1 shows the functional area and the key requirements

2.3 Constraints and Out-of-Scope Limitations

Each project is constrained by its limits and restrictions that outline its limits, thereby allowing for successful execution of project goals. As the systems develop, the changes and issues may arise that could lead to what is called additional features or enhancements in the future. Nevertheless, trying to satisfy all expectations within one cycle may distort the outcome of the project. It is crucial to specify the constraints and limitations of the project. In this context "constraints" refers to those factors or provisions that may impose restrictions on the implementation or development of the project under consideration. Limitations refer to those future improvements or additions that will not be a part of the project. Limitations help ensure that the project remains focused on its specified requirements and can be completed within the planned schedule while delivering the agreed-upon core functionality.

2.3.1 Constraints

- GPS accuracy may be reduced under Niah National Park's dense forest canopy. This affects the precision of recorded plant locations. It is a known environmental limitation rather than a system defect.
- IoT hardware for real-time environmental sensing may not be available for testing within the unit timeline. Sensor data may need to be simulated, and this assumption would be clearly flagged in the final report.

- Offline-first mobile sync must handle cases where multiple botanists edit the same record before reconnecting. The chosen conflict-resolution strategy will limit how complex simultaneous field edits can be.
- The unit timeline (a single trimester) limits the depth of testing possible for the vulnerability assessment and remediation cycle compared to a production deployment.

2.3.2 Out-of-Scope Features

- Public-facing visitor mobile app or QR-based educational content for ecotourism (the mobile app in this project is scoped to the Botanist role only).
- Facial or biometric recognition of park staff or visitors.
- Automated species identification from photographs using machine learning/AI.
- Integration with third-party government biodiversity databases outside SFC's own system.
- Physical deployment and long-term maintenance of IoT hardware in the field beyond the prototype/demo stage.
- (Optional) An innovative ground-truthing workflow beyond the objectives listed in section 2.2 is out of scope unless the team confirms with the supervisor that pursuing it is required or beneficial for full marks.

Type	Constraint / limitation	Implication for the project
Environmental	GPS accuracy reduced under dense forest canopy	Recorded plant locations are approximate. Known limitation, not a defect
Hardware	IoT sensors may not be available for field testing in time	Sensor data may be simulated. Assumption clearly flagged in the final report
Data integrity	Multiple botanists may edit the same record offline	Conflict-resolution strategy bounds how complex simultaneous edits can be
Timeline	Single trimester for vulnerability assessment and remediation	Depth of testing is limited versus production deployment
Out of scope	Public ecotourism app, external government DB integration, physical IoT deployment	Not delivered. Referenced only for context

Table 2 shows the constraints/limitation and the implication for the project

3 STAKEHOLDERS

A stakeholder is someone who has an interest in a business and can either influence the business or be affected by it. To ensure that the project will have clear definition of the people who will be involved in the project either directly or indirectly, below the table shows the stakeholders in this project and their interest.

Stakeholder	Role	Interest
Sarawak Forestry Corporation (SFC)	Client / business owner	Requires quicker and more accurate records of biodiversity in Niah National Park. Looking for less delay between field observations and availability of usable information. Acquisition of better instruments for conservation-related decision-making.
NeuonAI	Industry partner / commercialisation partner	A validated system it can eventually commercialise. Provides domain guidance on ground-truthing workflow and data standards.
Swinburne University of Technology Sarawak (COS30049)	Academic sponsor	The project complies with unit learning outcomes and engineering principles (SSDLC, testing, documentation). The tutor's role is that of a sponsor for the client.
Botanists / field officers	End user (mobile app)	An efficient and dependable offline method for the collection of field data without repetition of any paper and digital work.
Conservation officers	End user (web app)	An all-inclusive and searchable database of species, capability to approve/reject submitted field data, and the capacity to issue reports for the management and funding decisions.
Park visitors & local communities	Secondary/indirect user	View published species records through the public, read-only web portal, supporting biodiversity awareness and education. QR-based ecotourism content itself remains out of scope.
Project team (COS30049 Group 7)	Developers/providers	Deliver a working system meeting both the client brief and unit assessment requirements within the trimester.

Table 3 shows the stakeholders of the project, the stakeholders' role and their interest

4 POSSIBLE SOLUTION ANALYSIS

The meaning of a solution is method for solving a particular problem. However, there are different solutions that can be utilized for solving the same problem. The appropriateness of each solution will depend on such factors as project time, cost, and scope. Thus, looking at several potential solutions will allow the team to choose the right solution that meets the goals and objectives of a project.

To evaluate the solutions suggested in an impartial way, the team will apply three frameworks: Project Forces, SWOT Analysis, and KoST Analysis. The Project Forces framework analyzes important factors such as time, cost, and scope to explore the feasibility of the proposed solutions. The SWOT analysis evaluates each solution in terms of strengths, weaknesses, opportunities, and threats relevant to the solution as well as the potential business value of the solution. The KoST analysis provides an analysis of Knowledge, Skills, and Technology needed for the implementation of the suggested solutions.

Before deciding on our final approach, we looked at three different ways to solve SFC's biodiversity documentation problem.

- Option A: Just digitize the paper records. Keep the current manual process, still using paper forms and physical tags, but type the data into a spreadsheet or basic database afterward.
- Option B: Use an existing tool instead of building our own. Tools like ArcGIS Field Maps, Survey123, and iNaturalist already exist for field data collection, so we could configure one of these to fit SFC's needs rather than building custom software.
- Option C: Build our own system, tailored to SFC. Design and build a mobile app, web app, and IoT layer from scratch so the result matches exactly how SFC's botanists and conservation officers work, including QR tagging, offline recording, GPS, and sensor-based monitoring for endangered plants.

Evaluation Criterion	A: Digitise with generic tools (Excel, or SQL)	B: Adopting an existing platform	C: Tailored-built System
Works offline in the forest	Partial, but it depends on the chosen form tool’s offline support, and lacks a defined conflict-resolution strategy	Available in most mature platforms, typically as a paid tier feature	Native, offline-first and online-sync capabilities of the records and plant identification, tracking and tagging by design
Matches SFC's QR-tagging workflow	Not supported	Not supported without custom workarounds	Built to the specification and the requirements
System and data ownership	SFC owns the spreadsheet data but no structured system	Vendor-owned platform. SFC and NeuonAI have no ownership of the underlying system	Will be fully owned by SFC and NeuonAI
IOT-based threat monitoring	Not supported	Rarely supported, and not integrated with field records	Fully specified and controlled by the team, aligned to SSDLC
Security and RBAC control	Limited to whatever the chosen consumer tool provides	Determined entirely by the vendor's security model	Fully specified and controlled by the team, aligned to SSDLC

Evaluation Criterion	A: Digitise with generic tools (Excel, or SQL)	B: Adopting an existing platform	C: Tailored-built System
Cost	Lowest direct cost, but highest ongoing labour cost from manual consolidation	Recurring subscription cost, scaling with usage or seats	No license fees, just time consumption to build the apps (mobile app and the web app)
Deliverable within one semester (12 weeks)	Deliverable immediately, but does not resolve the underlying problem	Deliverable quickly, but with limited customisation depth	With the 13-week plan specified in Section 5, it should be feasible for the team given their existing knowledge of React/TypeScript, thus encouraging the maintenance of the highest possible level of industrial standard in the prototype development and report writing.

Table 4 shows Evaluation Criteria for each of the suggested solutions

4.1 Justification for Selecting Option C

Option A addresses the symptom of the problem (paper records) without resolving its cause (no offline-capable, QR-based, conflict-aware capture system) and does not extend to IoT-based protection at all. Option B is faster to configure but cannot meet SFC's QR-tagging and offline-sync requirements without substantial custom workarounds and would leave NeuonAI without a product of its own, which matters given its role as a commercialisation partner rather than a client. Option C requires the most implementation effort but is the only option that satisfies the offline, security, and IoT requirements SFC specified, while giving NeuonAI a system it can develop into a commercial product, comparable to its existing RoadPlus offering.

4.2 SWOT Analysis (Option A)

For completeness, the team also assessed Option A (digitising paper records with generic tools such as Excel or a basic SQL database) and Option B (adopting an existing field-data platform) on their own merits, using the same SWOT structure applied to Option C above.

4.2.1 Strengths

- Cheapest and fastest to start: no new infrastructure or unfamiliar technology, just data entry into tools the team already knows.
- Very low technical risk, since there is no framework or platform to learn and nothing new that can break.
- Could be actioned immediately, without waiting on a 13-week build.

4.2.2 Weaknesses

- No offline-first support or defined conflict-resolution strategy, so botanists in the field would still rely on paper forms until someone keys them in later.
- Cannot support QR-based tagging or GPS-linked records, so the scan-to-record objective in Section 2.2 cannot be met at all.
- No IoT or threat-monitoring capability whatsoever, leaving roughly half of SFC's stated requirements unaddressed.
- No built-in role-based access control or encryption, so it does not satisfy the SSDLC and security requirements in Section 2.3.1.

4.2.3 Opportunities

- Could serve as a temporary bridge process while Option C is being built, e.g. during Weeks 1-4 of the schedule in Section 5.2.
- Produces an initial digitised dataset that Option C's system could later import rather than starting from a blank database.

4.2.4 Threats

- Manual re-entry is highly error-prone, and transcription mistakes compound over time, undermining the accurate-records goal SFC asked for.
- Ongoing labour cost keeps growing as more records are collected, unlike Option C's one-time build effort (see the Cost row in the comparison table above).
- Leaves NeuonAI without any product to commercialise, which puts its continued interest as a partner at risk.

4.3 SWOT Analysis (Option B)

4.3.1 Strengths

- Mature, production-tested platforms (ArcGIS Field Maps, Survey123, iNaturalist) with existing offline support, in most cases deliverable within the semester.
- The vendor handles ongoing maintenance, updates, and security patching, reducing the team's own security-testing burden.
- Well documented and already used at scale elsewhere, which lowers short-term learning risk compared with building everything from scratch.

4.3.2 Weaknesses

- None of the evaluated platforms natively support SFC's QR-tagging workflow; making it work would need custom workarounds that are not guaranteed to succeed.
- SFC and NeuonAI would not fully own the underlying system, and depending on the platform, not all of their own data either, limiting how far it can be branded or extended later.
- IoT sensor integration and automated threat alerts are rarely supported out of the box and would still need a separate custom bridge, undermining the point of adopting a ready-made tool.
- Security and RBAC are fixed to the vendor's own model, which the team cannot audit or align to the SSDLC approach.

4.3.3 Opportunities

- A faster initial rollout could free up team time to focus on the IoT layer as a smaller custom add-on.
- Could double as a reference implementation the team benchmarks Option C's own usability and feature set against.

4.3.4 Threats

- Recurring subscription costs scale with usage or seats, an ongoing cost SFC would carry indefinitely, unlike Option C's one-time development effort.
- The vendor could change pricing, discontinue features, or alter API terms at any time, outside SFC's or the team's control.
- Leaves NeuonAI without a proprietary product to commercialise, undermining its role as a commercialisation partner.

4.4 SWOT Analysis (Option C)

With Option C selected as the recommended direction, the team assessed it on its own merits using a SWOT analysis, to surface risks worth planning for rather than discovering them mid-build.

4.4.1 Strengths

- Tailored-built to match SFC's actual QR-tagging, offline, and IoT workflow exactly, unlike Options A or B.
- SFC and NeuonAI fully own the data and system, with no vendor lock-in or ongoing subscription cost.
- Instead of being added later, security and role-based access control have been incorporated from the beginning.
- Constructed using reliable and proven technology (Supabase, React/React Native, MQTT) with a well-established community support.

4.4.2 Weaknesses

- Requires the team to design, build, and test every layer itself inside a single 13-week trimester which can be the most implementation-heavy option.
- No prior production track record, unlike adopting an established platform such as ArcGIS Field Maps.
- A 5-to-7-person student team has limited prior experience with IoT, and cybersecurity knowledge such as formal security testing.

4.4.3 Opportunities

- Gives NeuonAI a validated product to commercialise, similar to its existing RoadPlus offering.
- A working prototype could be extended to other national parks or genuinely adopted by SFC beyond the unit.
- Establishes an internal SSDLC and security-testing practice the team can reuse on future projects.

4.4.4 Threats

- IoT hardware (ESP32 sensors) may not be available in time, forcing reliance on simulated data for the final demo.
- Scope creep risk given the breadth of the system (mobile, web, IoT, and security) within a fixed trimester.
- Offline sync conflict handling introduces genuine technical risk if edge cases aren't tested thoroughly.

4.5 Project Forces Analysis (Force Field)

A force field analysis was used to weigh the forces pushing the team toward building Option C against the forces pushing back against it.

4.5.1 Driving forces (for building Option C)

- SFC's explicit need for offline, QR-based tracking that off-the-shelf tools don't fully support.
- NeuonAI's commercial interest in owning a purpose-built product rather than configuring someone else's.
- The unit's requirement to demonstrate full SSDLC practice and original architecture design.
- The team's existing familiarity with React/TypeScript across both mobile and web.

4.5.2 Restraining forces (against / risks)

- The single-trimester timeline limits the engineering depth achievable compared with configuring an existing tool.
- Team members have varying prior experience with mobile development, IoT, and security testing.
- Real IoT sensor hardware availability is outside the team's control.
- Effort for offline-sync conflict resolution and security remediation is easy to underestimate.

On balance, the driving forces outweigh the restraining forces: the restraining forces are manageable through mitigations already defined in the Section 7.1 Risk Register (simulated IoT sensors, a staged sync strategy, and a prioritised backlog), while the driving forces represent requirements Options A and B cannot meet at all.

4.6 KoST Analysis (Knowledge, Skills, Technology)

KoST assesses the team's readiness to deliver Option C across three dimensions: knowledge of the problem domain, the skills needed to build it, and the technology required to run it.

4.6.1 Knowledge

- The team understands SFC's field-data workflow (QR tagging, offline capture, conservation-officer review) from the client brief and the stakeholder analysis in Section 3.0.
- SSDLC and security-by-design principles are understood at a conceptual level. Practical experience running an OWASP ZAP scan will need to be built up before Week 11.
- Deep domain knowledge of plant taxonomy and conservation status is limited within the team which is mitigated by sourcing real records from GBIF and cross-checking against MyBIS/BRAHMS (Section 8.3) rather than relying on the team's own classifications.

4.6.2 Skills

- Existing familiarity with React/TypeScript covers both React Native (mobile) and Next.js (web), reducing the learning curve across two of the four mandatory delivery areas.
- The weakest skill areas for the team are IoT integration (MQTT, sensor ingestion) and formal security testing, in agreement with the Weaknesses of the team stated in Section 4.4.2 and Risks noted in the Risk Register Section 7.1 R2 and R4.
- These gaps are mitigated rather than ignored: simulated sensors let the team build and test the IoT pipeline without needing hardware expertise up front (Section 6.2), and Section 6.3 schedules security testing as a gated milestone rather than a last-minute task.

4.6.3 **Technology**

- The core stack (Supabase, SQLite, React Native, MQTT/Mosquitto, OWASP ZAP) is free or has a generous free tier, so cost is not a readiness blocker (Section 8.1).
- Real IoT hardware (ESP32 sensors) is the one technology dependency outside the team's control. The simulate-then-swap approach means the project can proceed on schedule without it (Section 2.3, Constraints).
- Hosting (Vercel/Render plus Supabase) and version control (GitHub) are already familiar to the team from prior coursework.

Overall readiness. Knowledge and Technology are largely in place. Skills is the main gap, concentrated in IoT and security testing, both already covered by mitigations built into the Initial Release Schedule (Section 5.3) and the Risk Register (Section 7.1) rather than left as unaddressed risk.

5 DELIVERABLES AND SCHEDULE

5.1 Deliverables

Documentation:

- Project proposal document
- System design document (ER diagram, architecture diagram, UI/UX wireframes)
- Security documentation: SSDLC plan, vulnerability assessment report, remediation log, re-assessment report
- Sprint reports (Sprint 1 and Sprint 2)
- Test documentation: test plan, unit/integration test cases and results
- User manual (botanist mobile guide + conservation officer web guide)
- Final report and presentation slides, with live demo
- (Optional) Innovative ground-truthing workflow write-up and evaluation

Software/ Hardware:

- Source code: mobile ground-truthing app, web knowledge system, backend API, IoT simulation & dashboard
- Working prototype covering all four mandatory areas

5.2 Schedule

Updated per client feedback: IoT foundational work now begins in Sprint #1 rather than Sprint #2, so that Sprint #2 is reserved for refinement, security testing, and integration rather than first-time building of major components.

Phase	Weeks	Deliverable
Proposal and Requirements	1–4	Project proposal document
System Design	5–6	Design document + wireframes + use case diagram
Core Backend & Database	6-7	Working API + DB
Mobile Ground-Truthing App	6-7	Functional mobile app
Web Knowledge System (+ public Visitor view)	8–9	Functional web portal, including read-only public pages
IoT Foundational Pipeline (moved from Wk10)	8–9	Simulated sensors, MQTT ingestion, basic monitoring dashboard
Security Testing	11	Security assessment report
IoT Refinement & Integration Testing	10–12	Stable full system, alert thresholds tuned, real ESP32 swap-in if available
Final Documentation & Demo	13	Final report + demo + presentation

Table 5 shows the schedule of the overall project throughout the project timeline

5.3 Initial Release Schedule

5.3.1 Product Backlog

To make sure that the main deliverables of this project can have complete functionalities by the end of the project, product backlogs have been listed below to show the features or functionality that will be developed for the prototype to be said as complete system. Each item in the product backlogs are important for the project in ensuring that the developed system will solve the issues that have been defined along with achieving the goals of the project.

No.	Item	Dependencies	Business Value (1 least - 10 most)	Release Schedule (Sprint #)
1	Set up Supabase project (PostgreSQL schema, auth, RBAC, storage)	N/A	10	Sprint #1
2	Deliver mobile offline-first field app: record capture (QR, GPS, camera)	1	9	Sprint #1
3	Deliver web knowledge system: species record CRUD + search	1	9	Sprint #1
4	Implement offline-to-cloud sync (SQLite to Supabase) with defined conflict strategy	2	8	Sprint #1
5	Public read-only Visitor view: browse and search published species reports (NEW)	3	6	Sprint #1
6	IoT sensor ingestion, monitoring dashboard and automated alerts (MOVED from Sprint #2)	1	7	Sprint #1
7	Add conservation officer review/approval workflow and searchable reporting	3	8	Sprint #1
8	IoT alert threshold tuning and real ESP32 swap-in (if hardware available)	6	5	Sprint #2
9	Complete security: vulnerability scan, remediation, re-verify before submission	2,3,4,6,7	8	Sprint #2
10	Final integration, system testing, documentation and demo	1–9	7	Sprint #2

Table 6 shows the product backlog of the items along with the release schedules and the dependencies for each of the item

5.3.2 Project Timeline

To give a clearer visual sense of how the 13 week schedule in Table 7 plays out, the figure below presents it as a Gantt chart. The early phases run sequentially: Proposal & Requirements occupies the first four weeks, both to give the team room to firm up scope with SFC and NeuonAI and because every later phase depends on that scope being settled, followed by System Design in Weeks 5-6 to produce the architecture and wireframes the build phase relies on.

From Week 7 onward the schedule follows the two-sprint structure set out in Section 6.3. Sprint #1 (Weeks 7-9) begins with the Supabase backend and database, since the mobile app, web app, and IoT service all depend on it being in place first. The Mobile Ground-Truthing App and Web Knowledge System are then built in parallel across Weeks 8-9 rather than one after the other, which is possible because the two apps share a common backend but not a common codebase dependency between them, the team's shared TypeScript/React Native and React/Next.js stack (Section 6.2) is what makes running both workstreams at once realistic for a team this size.

Sprint #2 (Weeks 10-12) shifts focus to the IoT monitoring layer, security testing, and integration. The IoT Module is scheduled for Week 10, immediately after the core backend and client apps are functional, so the sensor ingestion pipeline has real data to plug into rather than being built in isolation. Security Testing is deliberately placed at Week 11 rather than left until the end of the trimester: this gives the team a full week before Integration & Testing (Week 12) to remediate any vulnerabilities the OWASP ZAP scan finds and to re-verify them, in line with the SSDLC deliverable described in Section 7.2 and Appendix C. Week 13 closes the project with final documentation, the user manual, and the live demo.

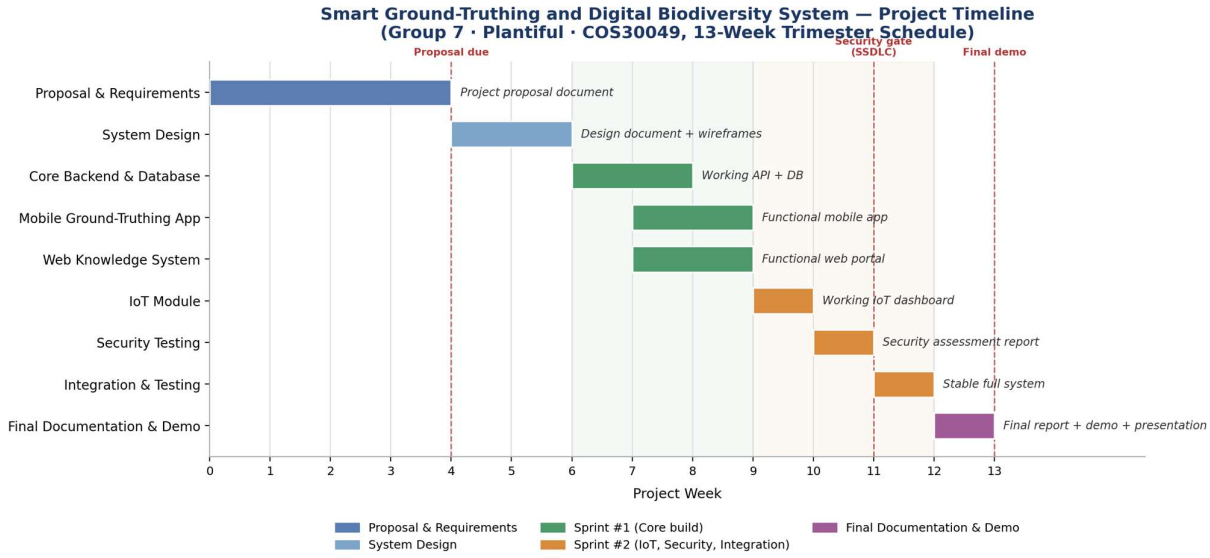


Figure 1 shows the visualisation of the fulfillment of the requirements and the schdule based on the project timeline

Three checkpoints in the schedule carry the most schedule risk and are marked on the chart accordingly: the Week 4 proposal deadline, the Week 11 security gate, and the Week 13 final demo. Of these, the Week 11 security gate is the one the team has the least slack around Risk R4 in Section 7.1 already flags that security vulnerabilities discovered late in the cycle carry high impact, and the one-week gap between Security Testing and Integration & Testing in this schedule is what gives the team room to act on that risk rather than discovering it too late to fix.

6 SOLUTION DIRECTION

6.1 Overview

This section explains the overall system and how its different users interact with it. Plantiful is a web-connected system used by four types of actors: Botanists, Conservation Officers, Admins, and public Visitors. Botanists use the mobile application to capture plant data in the field, including when working offline. Conservation Officers use the web application to review, manage, and publish biodiversity records. Admins manage user accounts and monitor the system's connected sensors. Visitors can access published species information through a public read-only interface without creating an account. All four actors ultimately interact with the same central database through the system's architecture, while role-based access controls ensure that each actor can only access the functions and information appropriate to their role.

The system is organised into three tiers: a Client Tier, an Application Tier, and a Data Tier. The Client Tier contains the mobile and web applications used by the different actors. The Application Tier contains the backend services responsible for authentication, role-based access control, record management, knowledge management, public access, and IoT data processing. The Data Tier provides the persistent storage used by the system, including Supabase for the central database, authentication, and file storage, SQLite for offline mobile data, and an MQTT broker for sensor telemetry. Most system requests follow a request-response pattern from the client through the application layer to the data layer. Sensor data follows a separate event-driven path because readings are continuously published through MQTT rather than requested by a user.

6.2 Client-Application-Data Architecture

Figure 1 shows the components and interactions across the three tiers. In the Client Tier, the Mobile App, built with React Native, is used by Botanists for field data capture and maintains a local SQLite store to support offline operation. The Web App, built with React and Next.js, provides the Conservation Officer Dashboard, Admin Dashboard, and public Visitor Portal. The Application Tier contains the Backend API monolith, which provides the Records Service, Authentication and RBAC Service, Knowledge System Service, and Public Read Service. The separate IoT Ingestion Service subscribes to sensor telemetry and evaluates configured alert thresholds. In the Data Tier, Supabase provides the managed PostgreSQL database, authentication, and storage, while SQLite provides local offline storage on mobile devices and the MQTT broker provides the communication layer for sensor telemetry.

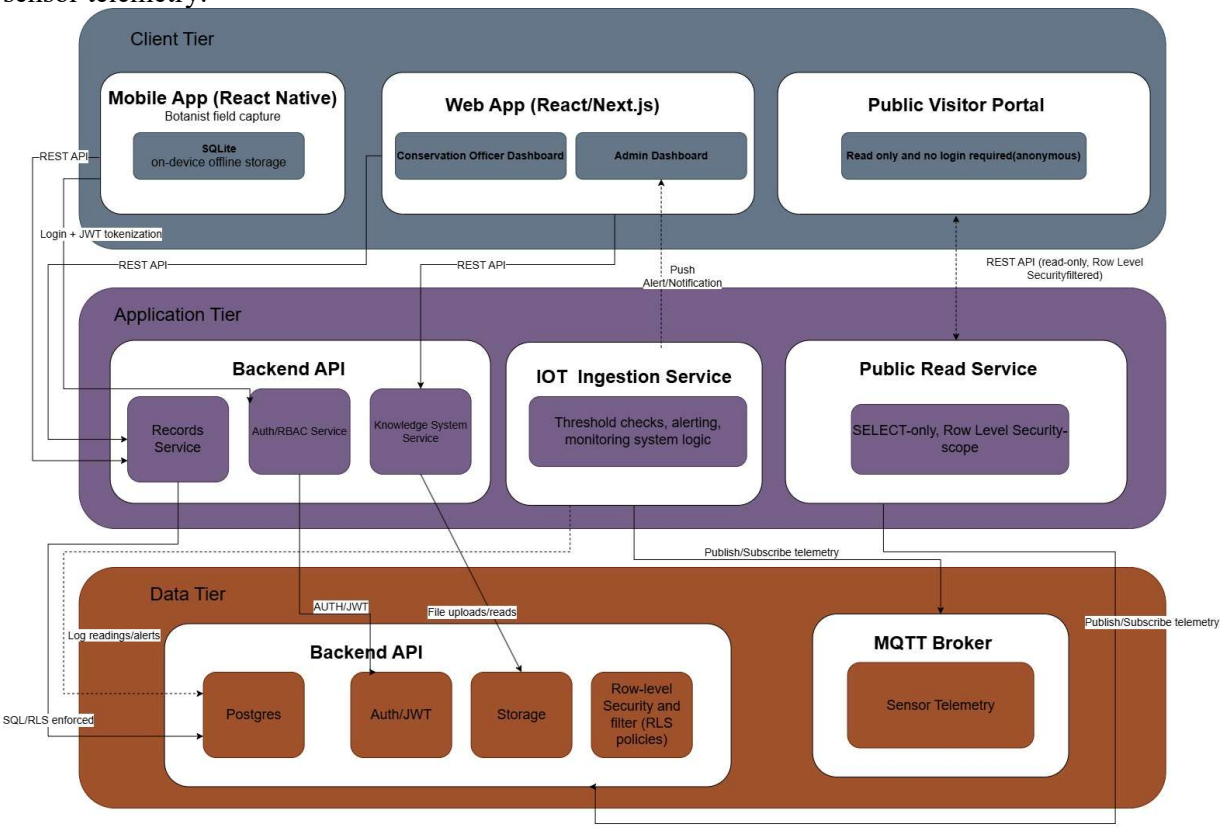


Figure 2 Plantiful system architecture. Client, application, and data tiers

Figure 2 shows the underlying data model. Visitors do not require a separate database table because they access the system anonymously and do not create user accounts. Instead, public visibility is controlled through an `is_published` attribute associated with species records. The Public Read Service only exposes species information and associated reports that have been marked as published. Database-level PostgreSQL Row-Level Security (RLS) provides an additional access control layer so that unpublished records remain inaccessible through the public interface. This ensures that work-in-progress records belonging to Botanists or Conservation Officers are not exposed to public users.

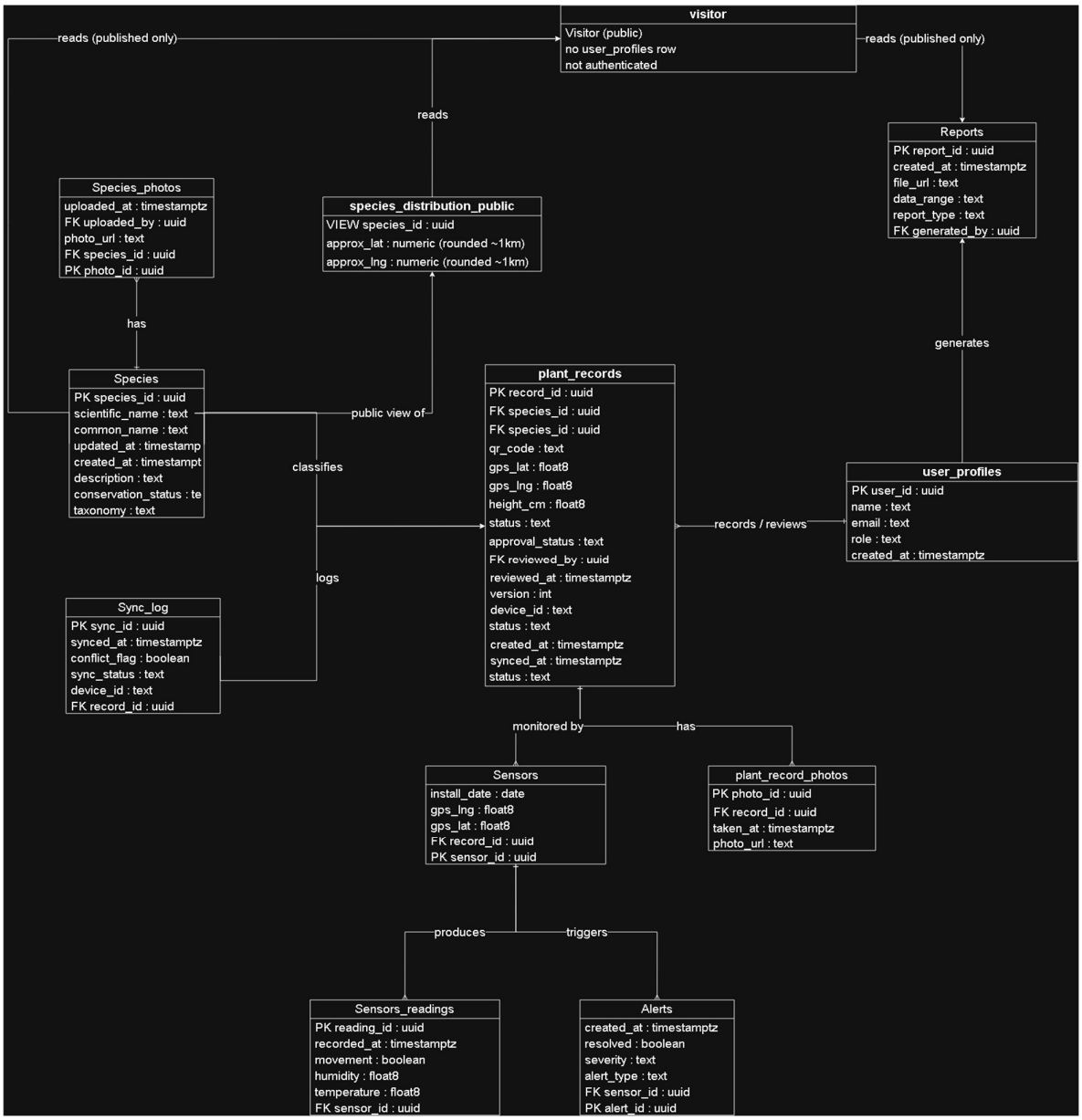


Figure 3 Plantiful data model (ER diagram)

6.3 Tier Summary

Tier	Components	Responsibility
Client tier	Mobile app, Web app (Conservation Officer + Admin dashboards, Public Visitor portal)	Field data capture by botanists (offline-capable). Species management, review, and reporting by conservation officers. User/sensor administration by admins. Read-only published-species browsing by the public, with no login
Application tier	Backend API monolith (incl. Public Read Service). IoT ingestion service	Business logic, authentication and RBAC, record CRUD, sync handling, public read-only queries.Sensor telemetry ingestion and alert evaluation
Data tier	Supabase (Postgres, auth, storage),SQLite (on-device), MQTT broker	Central source of truth and file storage, with RLS restricting public reads to published records.Offline buffer on the mobile device. Transport for sensor telemetry

Table 7 shows the components and the responsibilies of each of the tier

6.4 Sub-Component Technology Analysis

Section 4.0 compared the three overall options (digitise, adopt an existing tool, or build custom) and selected Option C. The sub-sections below repeat that same comparison process one level down, for each major sub-component inside Option C, so the choice of specific technology for each part is backed by the same kind of research and evidence.

6.4.1 Mobile Framework

Besides React Native, the team looked at Flutter and fully native development (Kotlin for Android, Swift for iOS). Flutter offers strong single-codebase performance but requires learning Dart, a language the team has no existing experience in. Fully native gives the best platform integration but means maintaining two separate codebases in two separate languages, which is not realistic for a 5 to 7 person team in 13 weeks.

Option	SWOT summary	KoST summary	Recommended
React Native	Strength: one language across mobile and web. Weakness: some third-party plugins lag behind platform updates.	Team already knows TypeScript/React. So, there would be low learning curve.	Yes
Flutter	Strength: strong single-codebase performance. Weakness/Threat: requires learning Dart from scratch.	No existing team knowledge of Dart. That means, the team would need ramp-up time the schedule does not have.	No
Native (Kotlin + Swift)	Strength: best performance and platform integration. Weakness: two codebases, two languages to maintain.	Team lacks Kotlin/Swift experience. So, the workload doubles for a small team.	No

Table 8 shows the SWOT and KoST summary and recommendation of each of the option for mobile framework

Justification of the chosen Mobile Framework:

React Native wins because it is the only option that lets the whole team share one language across the mobile app and (per Section 6.3.2) the web app, directly reducing the learning-curve risk flagged in the KoST analysis (Section 4.6.2), without giving up native camera/GPS access.

Owned by: Basill Agas Anak Heatley Rogers (Team Lead, Mobile App, Field Data Capture) and Muhammad Maqeel bin Muhammad Kahfi (Mobile App, Sync and Backend APIs), per the Project Team roles in Section 9.1.

6.4.2 Web Framework

Besides React/Next.js, the team looked at Vue/Nuxt and a simpler plain HTML/CSS/JavaScript build. Vue has a gentler learning curve but a smaller ecosystem and no shared language advantage with the mobile team. A plain HTML/JS build would be fast to start but hard to scale into a maintainable, role-based system with search, filtering, and reporting.

Option	SWOT summary	KoST summary	Recommended
React/Next.js	Strength: shares TypeScript/React knowledge with the mobile team. Weakness: heavier setup than a plain site.	Team already knows React. The team can directly reuse mobile team skills.	Yes
Vue/Nuxt	Strength: gentle learning curve. Weakness: smaller ecosystem, no shared skill benefit with mobile team.	Team has no prior Vue experience.	No
Plain HTML/CSS/JS	Strength: fast to start. Weakness: hard to scale into a role-based, searchable system.	Does not need new skills, but does not fit the system's complexity.	No

Table 9 shows the SWOT and KoST summary and recommendation of each of the option for Web Framework

Justification for the chosen Web Framework:

Next.js's route groups and middleware make it straightforward to serve the Conservation Officer Dashboard, Admin Dashboard, and the new Public Visitor Portal from one codebase while still gating each route by role, which neither Vue nor a plain build offers without significant extra work.

Owned by: Nathan Sebastian Learmonth (Web Knowledge System, Records) and Badrul Aliff Aiman bin Badrulmunirzaki (Web Knowledge System, Reporting and Maps), per the Project Team roles in Section 9.1.

6.4.3 Backend and Database

Besides Supabase, the team looked at Firebase/Firestore and a self-hosted Postgres instance with a custom-built REST API. Firebase is mature and has real-time features, but Firestore's NoSQL document model fits poorly with the relational, hierarchical species and taxonomy data this project needs. Self-hosting Postgres would give full control but means building and maintaining authentication, the REST API, and storage from scratch, a large extra workload on top of everything else in 13 weeks.

Option	SWOT summary	KoST summary	Recommended
Supabase (Postgres)	Strength: auto-generated REST API, auth, RLS, storage, free tier. Weakness: managed service, less low-level control.	Matches the team's relational data needs which need low setup effort.	Yes
Firebase/Firestore	Strength: mature, real-time sync. Weakness: NoSQL model fits poorly with relational taxonomy data.	Team has little Firestore-specific experience. Data modelling mismatch adds risk.	No
Self-hosted Postgres + custom API	Strength: full control. Weakness: must build auth, API, and storage from scratch.	Large extra workload the timeline does not allow.	No

Table 10 shows the SWOT and KoST summary and recommendation of each of the option for Backend and Database

Justification for the chosen backend and database:

Supabase's built-in row-level security is the deciding factor for the new public-facing requirement specifically. This way, it lets the team enforce "visitors can only read is_published = true records" as a single declarative policy at the database layer, instead of having to re-implement that filter correctly in every API endpoint by hand.

Owned by: Muhammad Maqeel bin Muhammad Kahfi (Mobile App, Sync and Backend APIs), since this role covers the shared backend both apps depend on, per Section 9.1.

6.4.4 *IoT Messaging Protocol*

Besides MQTT, the team looked at plain HTTP/REST polling and CoAP. HTTP polling is simpler to set up but is inefficient for continuous sensor data, either creating high network overhead or delayed alerts, and does not fit the event-driven nature of sensor telemetry described earlier in this section. CoAP is also lightweight and designed for constrained devices but has a smaller ecosystem and less mature dashboard and broker tooling than MQTT, adding implementation risk for a time-boxed student project.

Option	SWOT summary	KoST summary	Recommended
MQTT (Mosquitto)	Strength: lightweight, built for constrained devices and unreliable networks, wide ESP32 support. Weakness: needs a broker component.	Well-documented, mature libraries and team can learn it within the schedule.	Yes

Option	SWOT summary	KoST summary	Recommended
HTTP/REST polling	Strength: simple, no broker needed. Weakness: inefficient for continuous streaming data which result in delayed alerts.	Does not fit event-driven sensor data as explained earlier in this section.	No
CoAP	Strength: also lightweight, made for constrained devices. Weakness: smaller ecosystem, less mature tooling.	Higher implementation risk for a time-boxed project.	No

Table 11 shows the SWOT and KoST summary and recommendation of each of the option for IoT Messaging Protocol

Justification for the chosen IOT Messaging Protocol:

MQTT is the only option that matches how the sensors actually behave (publishing whenever they have data, not on request), and moving this pipeline earlier into Sprint #1 (Section 5.3) made the maturity and documentation of MQTT's tooling more important, since the team now has less schedule slack to debug an unfamiliar protocol.

Owned by: Gae Jayden MWINE (IoT-Based Plant Protection: sensor data pipeline, alert and threat detection logic, monitoring dashboard), per Section 9.1.

6.4.5 Public Visitor Access

This sub-component provides the public Visitor interface within the existing Next.js web application. Visitors access published species information through read-only routes without requiring an account. The team considered three implementation options: a read-only route group within the existing Next.js web application, a separate static public microsite such as Astro or plain HTML fed by a periodic data export, and a third-party headless CMS placed in front of Supabase for publishing species content.

Option	SWOT summary	KoST summary	Recommended
Read-only route group in the existing Next.js web app	**Strength:** reuses the same components, data-fetching, and deployment pipeline as the Conservation Officer/Admin views. RLS already gives it a safe data boundary. **Weakness**: public traffic shares infrastructure with the internal app, so needs basic rate-limiting.	Zero new framework to learn because of the team's existing Next.js and Supabase knowledge covers it directly.	Yes
Separate static public microsite (e.g. Astro) fed by data export	**Strength:** fully isolates public traffic from the internal app. **Weakness:** needs a new build tool, a new deployment target, and a sync/export job to keep it current.	Team has no existing Astro experience which adds a second deployment pipeline to maintain in 13 weeks.	No
Third-party headless CMS in front of Supabase	**Strength**: mature publishing workflows out of the box. **Weakness**: introduces a second system of record and a vendor dependency for a feature that is otherwise a simple read query.	Team has no CMS integration experience. Therefore, the extra vendor lock-in also runs against the ownership rationale in Section 4.1.	No

Table 12 shows the SWOT and KoST summary and recommendation of each of the option for Public Visitors

Justification for the Public Visitor flow and system:

The read-only route group within the existing Next.js web application is the most suitable option because it reuses the team's existing framework, components, deployment pipeline, and Supabase integration. It does not require an additional framework, deployment target, or third-party vendor. Row-Level Security provides the required database-level boundary for restricting public access to published records, while the Public Read Service provides a thin application layer for retrieving those records. This approach keeps the public interface within the existing architecture while limiting the additional implementation and maintenance overhead required during the 13-week project.

Owned by: Ashley Wallen Anak Winston (Integration, PM & Documentation), extending their integration role to cover the new public route group and its rate-limiting, alongside Nathan Sebastian Learmonth (Web Knowledge System, Records) for the shared Next.js components.

6.5 Key Designs

Mobile framework: React Native is used so the whole team can work in one language, TypeScript, across both the field app and the web knowledge system. Using a single language allows the team to share QR, GPS, and camera handling logic and enables every member to contribute to either application without maintaining separate Dart and TypeScript development workflows. The mobile app provides an offline-first local SQLite store, QR scanning, GPS, and camera functionality as required. Flutter was considered as an alternative but was set aside because it would introduce a separate programming language and has weaker alignment with the chosen web stack.

QR strategy: Each QR code encodes only a stable record ID, which is resolved against Supabase when online or against a locally synchronised cache when offline. This allows records to be corrected centrally without requiring physical QR tags to be reprinted.

Sync conflict resolution: Sync conflict resolution is implemented in stages. Last-write-wins, applied after Conservation Officer approval of the record, provides the initial working sync pipeline. A manual conflict queue routed to the Conservation Officer's web dashboard is planned as an enhancement if development time permits. If the conflict queue cannot be completed within the available schedule, the system will retain last-write-wins as the fallback mechanism.

Central and offline database split: Supabase, using managed PostgreSQL with an auto-generated REST API, authentication, row-level security, and file storage, serves as the central database. SQLite on the mobile device provides the offline-first local store. REST provides the transport between the mobile application's local SQLite store and Supabase when connectivity is restored and is also used by the web application and IoT service to communicate with Supabase.

IoT approach: Simulated sensors publish over MQTT using a fixed topic and payload contract so that the complete pipeline, including the broker, ingestion service, database, dashboard, and alerting, can be built and tested end to end. Real ESP32 hardware can then publish to the same topic and schema without requiring changes to downstream components. IoT foundational work begins in Sprint #1, providing additional time for integration and testing before the security gate.

Public visibility approach: A Conservation Officer determines when a species record is ready for public access by setting `is_published = true` as described in Section 6.3.5. Records are not visible to Visitors by default. The Public Read Service only issues read queries for records where this flag is enabled, with the same restriction enforced at the database layer through Supabase Row-

Level Security. This provides two independent access-control layers, with both the application logic and database policy required to permit a record to appear in the public interface.

Security approach: Threat modelling and secure design decisions are incorporated during the architecture and design stage. These decisions are subsequently verified through an OWASP ZAP scan, remediation, and a reassessment pass in Week 11, in accordance with the Secure Software Development Lifecycle.

6.6 Development Process, Standards and Conventions

Beyond the technical architecture, the team follows a consistent set of process standards, so the codebase and delivery stay coherent across six contributors.

Development approach. Agile is conducted as two sprints that have been defined in Section 5.3 (Sprint #1 and Sprint #2) along with re-prioritisation of the backlog at every sprint boundary with risk register (Section 7.1) updates performed simultaneously.

Version control. Git and GitHub (Section 8.1), using short-lived feature branches per backlog item, merged into main via pull request with at least one other team member's review before merging.

Coding standards. Using TypeScript for both mobile (React Native) and web (Next.js) applications, maintain a coherent ESLint and Prettier configuration for conformity during development, sharing type definitions and schemas for various types of records between both apps where applicable.

Documentation standards. A README per major component (mobile app, web app, backend API, IoT service) covering setup and key architecture decisions, inline comments on non-obvious logic such as sync conflict handling and alert thresholds, and this proposal plus the system design document kept as the source of truth for architecture decisions.

Review and testing discipline. The unit and integration tests defined in Section 7.2 run before a pull request is merged, and the OWASP ZAP scan and remediation cycle in Section 6.2 gate the security-testing milestone rather than being left to the final week.

6.7 The Four Core Workflows

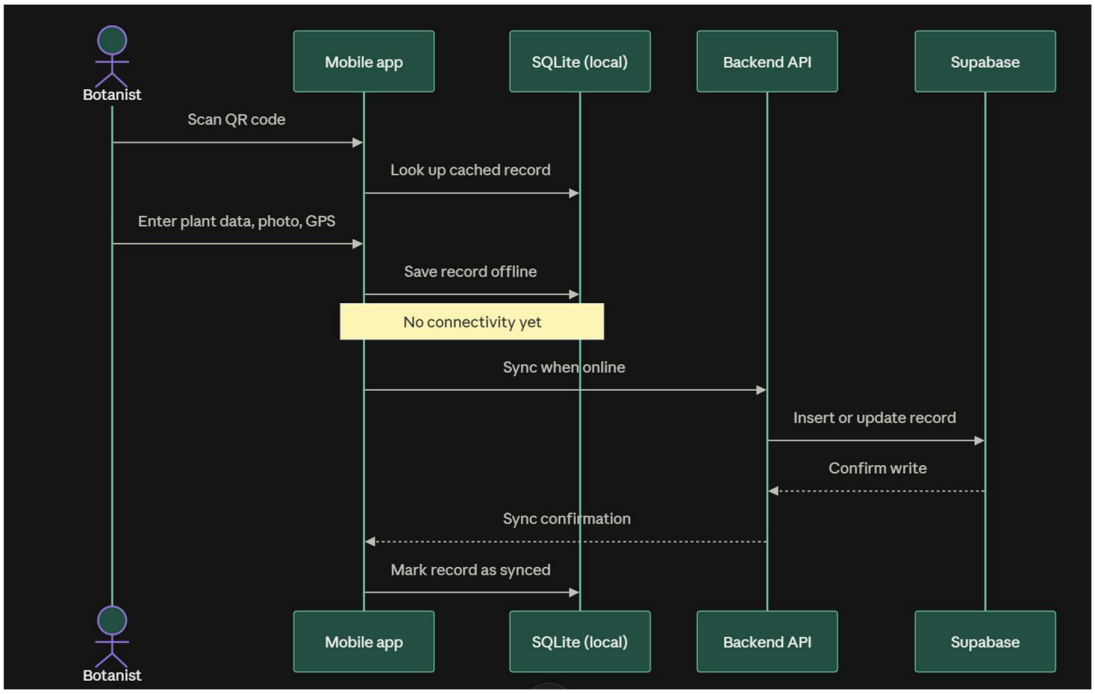


Figure 4 Botanist field capture flow. Offline capture, QR, sync, and review.

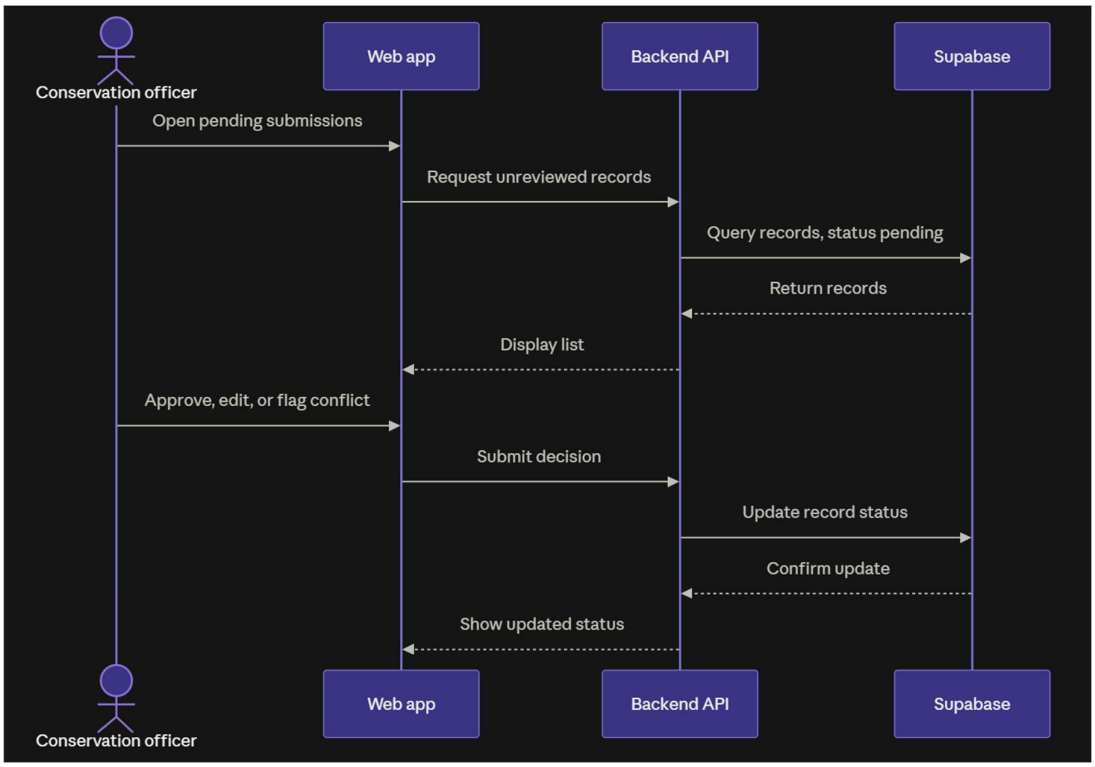


Figure 5 Conservation officer review and approval flow

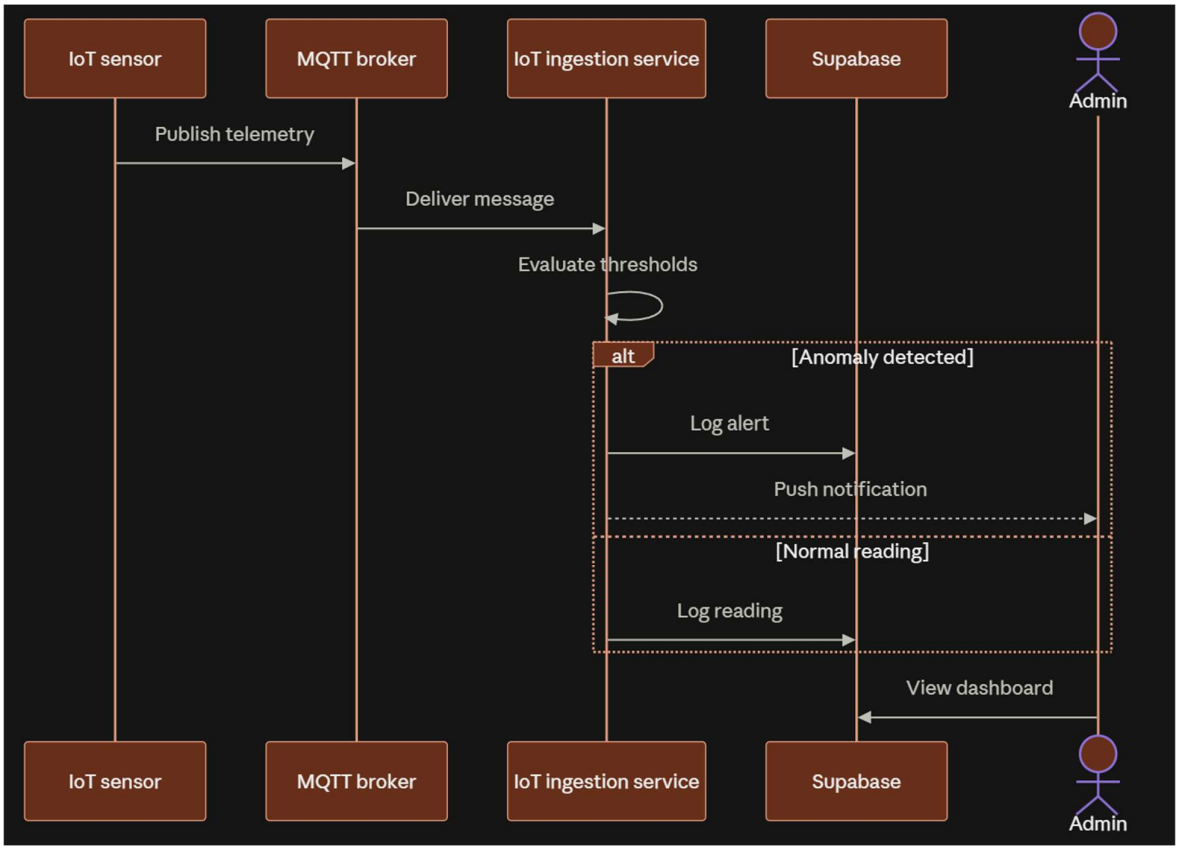


Figure 6 Admin system administration flow

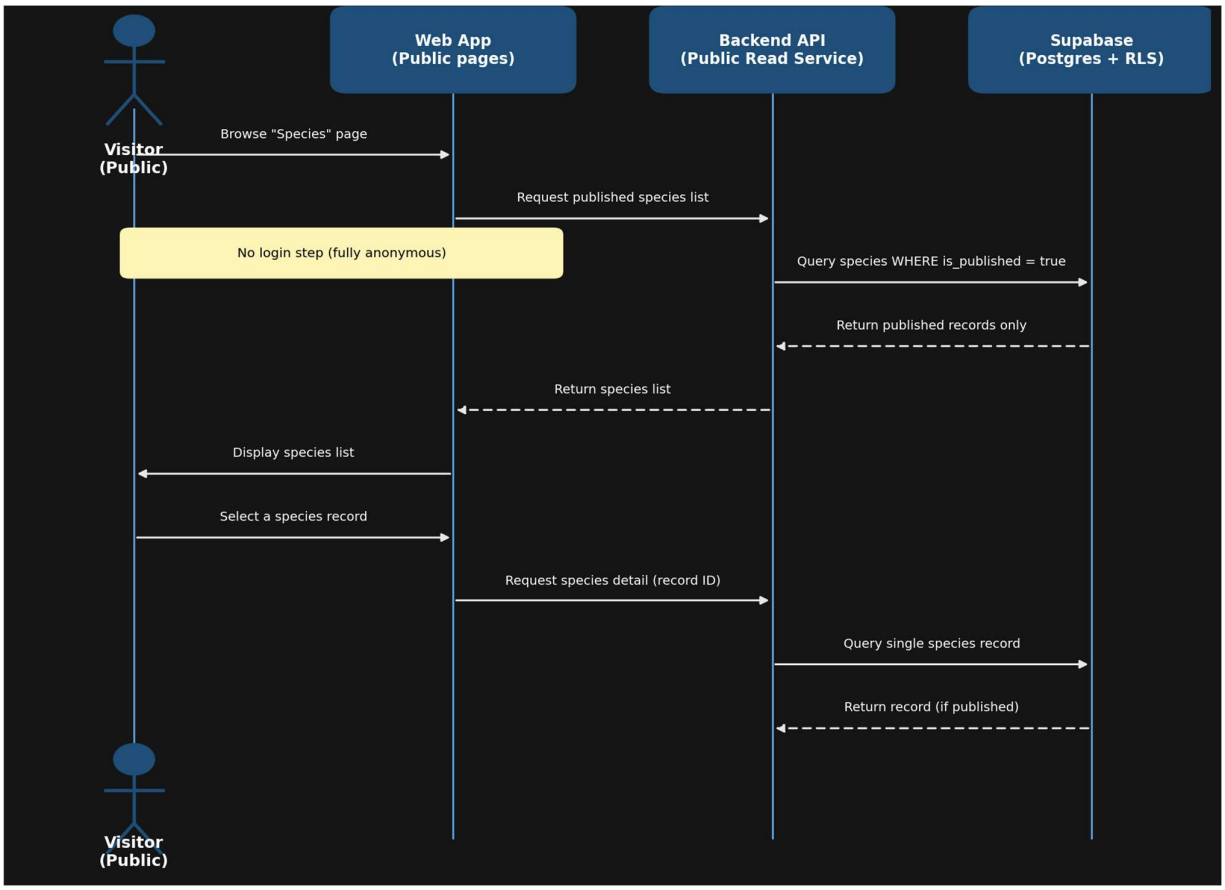


Figure 7 Public Visitors system flow

7 QUALITY MANAGEMENT

Quality can be analyzed according to five dimensions derived from the expectations of both SFC, NeuonAI, and the end-users of the system. Below are the explanations of each of them.

Functional quality. The system performs the core processes perfectly well from QR scanning to gathering data in offline mode, synchronizing to the central server, and going through review and approval of the specialists without any loss or damage of the data.

Data quality. The input species records are precise, complete, and consistent including absence of duplicates or empty fields where previous field input is obligatory.

Security quality. The access level management is functioning correctly limiting creation, updating, and approving of the records according to the rules described in the SSDLC document, and correctly limiting Visitors to published, read-only data only.

Usability. The mobile app can be used by the botanists and field officers regardless of the low level of connectivity, usage of gloves, or work under direct sunlight with the provided manual. The public view is usable by visitors with no prior account or instructions needed.

Reliability. The offline-first system is up to the task in real field conditions and works correctly employing safe data synchronization.

Quality Atttribute	Metric	Target	How Measured
Reliability (Offline sync)	Sync success rate	More than 95% of test submissions sync without data loss after reconnecting	Field/offline simulation testing (Week 12, Integration & Testing phase)
Functional performance	QR scan to record time	Less than 5 seconds average	Mobile app performance testing
Security	Critical/high vulnerabilities after remediation	No security vulnerabilities remaining	OWASP ZAP re-assessment report (Week 11 per SSDLC deliverable)
Code quality	Test coverage on backend API and mobile sync module	More than 80%	Unit or integration test suite results
Usability	Task complete rate (unassisted)	More than 90% across 5-user field-officer walkthrough	Moderated usability test using the draft user manual
Data integrity	Mandatory field enforcement	100% of required fields validated before sync	Backend validation testing
Public access correctness (NEW)	Unpublished-record leak rate via the public view	0%, no unpublished record ever returned to an unauthenticated request	RLS policy test + automated API test suite hitting the Public Read Service directly

Table 13 shows the how the metric and taget are measure for each of the quality attribute

7.1 **Risk Register**

#	Risk	Likelihood	Impact	Mitigation	Owner
R1	Data loss during offline-to-cloud sync	Medium	High	Byte-safe sync validation, conflict-resolution strategy, retry with last-write-wins fallback	Mobile App (Sync & Backend APIs)
R2	IoT hardware unavailable within the trimester	Medium	Medium	Use simulated sensors over MQTT with an identical topic/payload contract so real ESP32 devices can be swapped in without downstream change	Integration, PM & Documentation
R3	Poor or no connectivity in the field	High	Medium	Offline-first SQLite store with automatic background sync once connectivity returns	Mobile App (Field Data Capture)
R4	Security vulnerabilities discovered late in the cycle	Medium	High	SSDLC-aligned OWASP ZAP scan in Week 11 with remediation and re-assessment before submission	Shared by Team
R5	Scope creep against the 13-week timeline	Medium	Medium	Prioritised Initial Release Schedule (Sprint #1/#2) with out-of-scope features tracked and re-baselined	Team lead
R6	Duplicate or inconsistent species records	Medium	Medium	Unique QR record ID, mandatory-field validation, and the review/approval workflow	Web Knowledge System (Records)
R7	Publicly exposing unapproved or sensitive data (e.g. precise GPS of an endangered species) through the new Visitor view (NEW)	Low	High	Visitor view is opt-in per record via is_published, enforced twice in application logic and Supabase RLS. Conservation officers can withhold precise coordinates from published reports on a per-species basis	Web Knowledge System (Records) / Public Visitor Access Layer

Table 14 shows the likelihood and impact of each risk and the mitigation to resolve/reduce the risk

7.2 **Acceptance Criteria and Test Strategy**

The system is accepted only when it meets the measurable targets in the Section 7.0 quality matrix. Verification is layered across Weeks 11 to 13 in three ways:

Acceptance criteria: Every requirement maps to a measurable target in the quality matrix. A criterion passes only if its metric is achieved in the verification run.

Test strategy: Unit tests cover the backend API and the mobile sync module. Integration tests cover offline sync to Supabase, MQTT ingestion, and the Public Read Service's RLS filtering. System acceptance tests cover the offline-to-online sync round-trip, QR scan to record, role-based access control checks, and an explicit check that no unpublished record is ever returned to an unauthenticated Visitor request.

Verification evidence: This includes the OWASP ZAP report with a remediation log and re-assessment in Week 11, the integration results and sync-success-rate simulation report in Week 12, and the usability walkthrough results in Week 13.

8 RESOURCES

This section outlines the materials, manpower, and equipment used to complete this project. It is separated into Human Resources and Non-Human Resources, Software/Tools, Hardware (IOT purposes) and Plant Data Sources. Human resources are intangible resources that originate within and consist of personal qualities or characteristics.

8.1 Software / Tools

What it's for	Tool	Why
Mobile app	React Native	One TypeScript codebase for the offline field app (camera, GPS, QR scanning)
Web app	React / Next.js + Tailwind	The knowledge system conservation officers use
Database & backend	Supabase (PostgreSQL)	Central database, auto-generates our APIs, handles login/security, stores files
Offline storage	SQLite	Stores data on the phone before it syncs
QR codes	qrcode / zxing	Generating and scanning QR codes
Maps	Google Maps / Mapbox	GPS tagging and distribution maps
IoT	MQTT (Mosquitto) + InfluxDB	Sensor data pipeline and storage
File storage	Supabase Storage	Plant photos and documents
Security testing	OWASP ZAP or Burp Suite	Checking for vulnerabilities (needed for SSDLC)
Version control	Git + GitHub	Support coding collaboration with the team
Planning/design	Figma, Word, draw.io	Wireframes, diagrams, writing docs

Table 15 shows the software tools that are going to be used for building the prototype

8.2 Hardware

What	Why
Smartphones with GPS + camera	To test the mobile app properly
Laptops	For actually building everything
IoT sensors (or simulated)	For the plant protection module, real ones if we can get them, otherwise we simulate the data
Patchy/offline internet for testing	To make sure offline sync actually works like it would in a forest

Table 16 shows the hardware tools that are going to be used for building the prototype

8.3 Plant Data Sources

Source	What we're using it for
GBIF	Sample plant records for Sarawak/Borneo to seed our database
MyBIS / Forest Dept Sarawak (BRAHMS)	Double-checking conservation status info if we can get access

Table 17 shows the plant data sources to obtain the information of the plants in Sarawak

9 APPROVAL SIGNATURES

9.1 Project Team

#	Name	Student ID	Signature	Roles
1	Nathan Sebastian Learmonth	102782258		Web Knowledge System (Records)
2	Badrul Aliff Aiman bin Badrulmunirzaki	102778273		Web Knowledge System (Reporting & Maps)
3	Muhammad Maqeel bin Muhammad Kahfi	102782384		Mobile App (Sync & Backend APIs)
4	Ashley Wallen Anak Winston	105806559		Integration, PM & Documentation
5	Basill Agas Anak Heatley Rogers	102778888		Team Lead, Mobile App (Field Data Capture)
6	Gae Jayden MWINE	104393610		IoT-Based Plant Protection (sensor data pipeline, alert/threat detection logic, monitoring dashboard)

9.2 **Project Sponsor**

Tutor's name (on behalf of the client)	Signature
Lee Sue Han	

10 REFERENCES

1. GBIF (Global Biodiversity Information Facility) n.d., GBIF, viewed 24 September 2026, https://www.gbif.org
2. InfluxData n.d., InfluxDB documentation, viewed 24 September 2026, https://docs.influxdata.com
3. MQTT n.d., MQTT specification, viewed 24 September 2026, https://mqtt.org
4. MyBIS (Malaysian Biodiversity Information System) n.d., MyBIS, viewed 24 September 2026, https://www.mybis.gov.my
5. OWASP (Open Web Application Security Project) n.d., OWASP ZAP desktop user guide, viewed 24 September 2026, https://www.zaproxy.org/docs/desktop/start/
6. React Native n.d., React Native documentation, viewed 24 September 2026, https://reactnative.dev
7. Sarawak Forestry Corporation n.d., Niah National Park conservation and research resources, viewed 24 September 2026.
8. SQLite n.d., Query language understood by SQLite, viewed 24 September 2026, https://sqlite.org/lang.html
9. Supabase n.d., Supabase documentation, viewed 24 September 2026, https://supabase.com/docs

11 APPENDIX A: GLOSSARY OF TERMS

Term	Definition
Botanist	Field researcher who captures plant records using the mobile app
Conservation Officer	Staff member who reviews, approves and manages biodiversity records in the web knowledge system
SFC	Sarawak Forestry Corporation
SSDLC	Secure Software Development Lifecycle
Supabase	Managed Postgres backend providing database, authentication, and storage
MQTT	Lightweight publish/subscribe protocol for IoT sensor messaging
RBAC	Role-Based Access Control
QR	Quick Response code

12 APPENDIX B: OFFLINE-FIRST SYNC AND CONFLICT RESOLUTION RULES

- Each record has a globally unique stable ID, encoded in its QR tag.
- On capture offline, the record and photos are stored in local SQLite with a pending-sync status.
- On reconnect, records sync in timestamp order. A record is marked synced only after the server acknowledges it.
- If two botanists edit the same record offline, last-write-wins is used by default. A manual conflict queue is available to conservation officers as an enhancement.

13 APPENDIX C: SECURITY TESTING EVIDENCE (SSDLC)

- Week 11. OWASP ZAP baseline scan results
- Week 11. Remediation log (vulnerability → fix → verification)
- Week 11. Re-assessment report showing 0 critical/high findings remaining.
- Week 12. Integration test results (offline sync, MQTT ingestion, RBAC checks)